Naming service

Common task in distributed applications: find communication partner, component or service by a given name

- Find entity by its name
- Entities: files, computers, printers, data records, objects etc.
- Operations
  - bind a name
  - name-based lookup
  - maintain bindings
- Examples
  - DNS (Domain Name Service)
  - WINS (Windows Internet Name System)
  - Filesystem

Solving a common task in distributed systems, naming service can help us to locate our communication partners, and find components and services using arbitrarily chosen names. The naming service frees us from hardwiring the physical location of components and services in the code, and hence fosters more flexibe and easy-to-configure designs, as well as easy-to-read, easy-to-maintain program code.

The main idea is as follows. We assign names to the components/services in our application according to the logical structure of the application. We register the name - physical location binding in a special service, the naming service, the physical location of which is known to all components. When a component wants to find another one by name, it can contact the naming service, and ask for the physical location registered for that name.

We can find naming services in many areas, providing name-based lookup for different kinds of entities. We can find computers on the internet using names like `www.elte.hu`, which is made possible by the Domain Name Service. We can identify files in the filesystem of our computer by their names in a very similar manner.

The operations performed in a naming service are as follows: create a binding for the name and the entity, and register it into the service; find an entity using its bound name; and finally, update a binding in the service.

Java Naming and Directory Interface

- Access to naming service
- Finding communication partner, component or service
- `rmiregistry`, `tnameserv`
- Common interface for naming and directory services

- Part of the Standard Edition of the Java Platform

JNDI provides a nice interface to naming services. We typically use it in distributed Java applications for locating our communication partners. We have already seen uses of naming services for RMI and CORBA, namely the use of `rmiregistry` for RMI, and `tnameserv` for Java-IDL. While they have similar purpose and similar functionalities, they are different naming service implementations. JNDI gives a uniform interface for such services, so that we can abstract over the concrete implementation of the naming service in our Java application. This makes our application logic less dependent on middleware services. Being part of Java EE, JNDI was an important API for enterprise application programmers. With time, JNDI became part of the Standard Edition as well.

Directory service

- Extension to naming service
- Attributes can be assigned to entities
- Additional operations
  - bind attributes to the entities
  - maintain binding
  - attribute-based lookup
- Examples
  - LDAP (Lightweight Directory Access Protocol)
  - NIS (Network Information System)

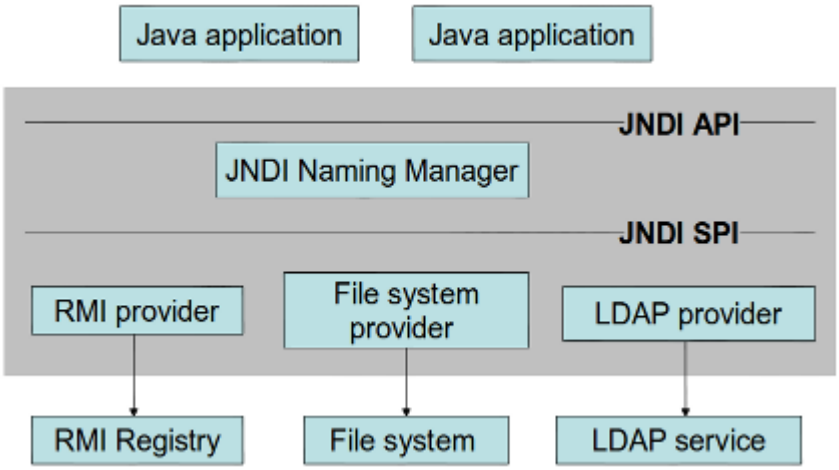
As the name suggests, JNDI is an interface not merely for naming services, but for directory services as well. Directory services are generalization of naming services. They allow the binding of attributes to entities, and support the lookup operation by the value of attributes as well. A well-known directory service is LDAP.

JNDI specification

- Application Programming Interface
  - JNDI client library
  - only this part is seen by Java programs
- Service Provider Interface
  - any provider can make its service available through JNDI

The JNDI specification is intended to provide single unified interface for using naming services, as well as an interface for naming service implementation to integrate into systems. One part of the specification is for service providers (SPI), and another part is for service consumers (API). Java application programmers need to consider the API. The SPI is only interesting for developers of middleware services.

Consuming services through JNDI



The figure demonstrates the structure of the JNDI framework and its interaction with Java applications. We will focus on the use of the JNDI API. The service provider interface will enable middleware developers to integrate their naming service implementation with other Java platform technologies easily, as well as to enable *federations*. When two or more naming services (even with different implementations) can be combined into a federation, one can access the services of all when executing a single lookup. We shall come back to this later on.

Packaging

- JNDI API
  - `javax.naming`
  - `javax.naming.directory`
  - `javax.naming.event`
  - `javax.naming.ldap`
- JNDI SPI
  - `javax.naming.spi`

Java programmers concern primarily the `javax.naming` package, which contains the most important functionalities of the naming API. The `directory` subpackage contains the API for the directory service, and the `event` subpackage is for working with events, such as changes to bindings during the service. The `ldap` subpackage provides special support for the wide-spread LDAPv3 protocol. Finally, the `spi` subpackage describes the SPI, and hence it is logically not part of the API.

Architecture

- Forest structure
- Root node of a tree: an URI
- Entity: a path
- Node: list of bindings
  - Name and object
  - Attributes (in directory service)
  - An object can be a node (hierarchy)
- Within a node: list of event handlers

The name space managed by a naming service has a forest structure in the JNDI approach. The trees in the forest are rooted using an URI. Each entity can be identified as a path in a tree.

The nodes of a tree contain name bindings for the children. A child can be an entity made available by the service, or another node: thus providing the hierarchy. In the case of a directory service, a node may contain attributes as well. Furthermore, we can attach event handlers to nodes, which are called automatically by JNDI when a specific event occurs (such as a change in a binding).

Node representation: Context

`javax.naming.Context` interface

```
void bind(String name, Object obj);
void unbind(String name);
void rebind(String name, Object obj);
void rename(String oldName, String newName);
Object lookup(String name);
/* ... */
```

A node in a tree of the JNDI forest is represented with the `java.naming.Context` interface. Among many others, this interface provides operations for manipulating bindings. You can establish and destroy a binding, you can change the object bound to a given name, and you can change the name of a registered object. Finally, the `lookup` method can be used to retrieve the object bound to a given name.

Initial context

- The root node of a tree
- `javax.naming.InitialContext` class

- Gaining access to the first `Context` object

```
Context ctx = new InitialContext();
DataSource ds = (DataSource) ctx.lookup("jdbc/DB");
```

- JNDI must be initialized

In order to get our first reference to a valid `javax.naming.Context` object, we can instantiate the `javax.naming.InitialContext` class, which implements the `Context` interface.

An object of that class may correspond to the root of a tree in the JNDI. The `InitialContext` object needs to know how to work with the used naming service implementation, therefore it requires initialization information. One can provide this information in different ways, as we will see soon. In the above example, we assume that JNDI is initialized before calling the no-arg constructor of the `InitialContext` class. Once the object is created, we can perform the `Context` operations, for instance we can call the `lookup` method to find a JDBC data source called `jdbc/DB`.

Closing the context

- Resource management in Java
  - `close` method
  - Release resources early
- try-with-resources not applicable
- Explicit `try-finally` required

```
Context ctx = new InitialContext();
DataSource ds;
try {
    ds = (DataSource) ctx.lookup("jdbc/DB");
} finally { ctx.close(); }
```

A `Context` object represents a resource; as with other resource objects, the standard way to support early-enough release of unused resources is the introduction of a `close` method. Calling it as soon as possible will help economize resources. This method is idempotent, but no other methods should be called on the context object after calling `close`.

Note that the JNDI context does not implement the `java.lang.AutoCloseable` interface, therefore the try-with-resources statement is not applicable for them. We need to use traditional, explicit try-finally to close a JNDI context.

JNDI providers

- File System Service Provider for JNDI
  - fscontext.jar, providerutil.jar
- RMI Registry Service Provider for JNDI
  - part of JDK (JNDI interface + `rmiregistry`)
- CORBA naming service
  - part of JDK (JNDI interface + `tnameserv`)
- LDAP
  - part of JDK (JNDI interface)
  - e.g. OpenLDAP, `www.openldap.org`

The JNDI SPI provides the means by which developers can write different naming and directory service providers and make them available to any service consumer. The list above contains some of existing JNDI service provider implementations. Some of the providers, such as rmiregistry and tnameserv, are already part of the JDK. Other JNDI providers are contributed separately. For instance, we will use the File System Service Provider for JNDI to demonstrate some functionalities of a JNDI service. This provider can be freely downloaded from Java Platform Technology Downloads, Java Naming and Directory Interface package (at `http://www.oracle.com/`). This JNDI provider contains two JAR files, which must be added to the classpath.

If we run `rmiregistry` or `tnameserv` (part of the JDK), we can connect to them using JNDI and the service providers shipped with the JDK; there is no need to add anything to the classpath. The JDAP Service Provider is also part of the JDK. One can install an LDAP implementation (for example OpenLDAP), and access it through JNDI, again without adding anything to the classpath.

Initial context parameters

- Factory class for initial context (JNDI provider specific)
  - in the case of the File System Service Provider: **com.sun.jndi.fscontext.FSContextFactory**
  - in the case of the RMI registry Service Provider: **com.sun.rmi.registry.RegistryContextFactory**
- Initial context URL
  - in the case of the File System Service Provider: defaults to **file:///**
  - in the case of the RMI registry Service Provider: defaults to **rmi://localhost:1099**

JNDI must be initialized prior instantiating the initial context. When used within a Java Enterprise application, the initial context is typically set up by the application server. The use of JNDI within application servers will be discussed later.

In this lesson we will demonstrate the usage of JNDI in standalone applications.

The initialization of JNDI usually involves the setting of two properties. The first property specifies which service provider to use. We need to give the name of the service provider specific factory class, which will be used for creating an initial context that can connect to the respective service implementation. This is a class with a name ending in `ContextFactory`.

The other property can describe the root of a tree in JNDI which will be represented by the initial context to be created. In the case of the file system service provider, it needs to refer to a directory in the file system (the `file:` URI is to be used). In the case of the RMI registry, for instance, the initial context URL should be given as a `rmi:` URI, and it should specify the machine and the port where the `rmiregistry` process that is intended to be used is accessible.

Passing JNDI parameters - 1

```
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.naming.NamingException;
import java.util.Hashtable;
public class InitWithHashtable {
  public static void main( String[] args ) throws NamingException {
      Hashtable<String,String> env = new java.util.Hashtable<>();
      env.put( "java.naming.factory.initial",
               "com.sun.jndi.fscontext.FSContextFactory" );
      env.put( "java.naming.provider.url", "file:///home/kto/" );
      Context ctx = new InitialContext(env);
      try { System.out.println( ctx.getEnvironment() ); }
      finally { ctx.close(); }
  }
}
```

There is a simple way to pass the properties to JNDI: one can put the property-value bindings into a `Hashtable`, and hand it over to the `InitialContext` constructor. For the sake of the example, we merely print out the very same environment, but now obtained from the context by calling the `getEnvironment` method. When something goes wrong with the use of the naming service, `javax.naming.NamingException` (or some subclass) is thrown, which is - obviously - a checked exception.

Note that we have specified the name of the factory class as a string literal. This name will be used by the class loader to look up the class in the classpath, and load it dynamically. Therefore we need to set the classpath properly when running this program.

How to compile and run?

Assuming a Linux machine, and that the two JARs are in the working directory...

```
javac InitWithHashtable.java
java -classpath fscontext.jar:providerutil.jar:. InitWithHashtable
```

There is no static reference to the contents of the JAR files in the code. Hence it is not necessary to include the JARs in the classpath for compilation, only for execution.

The above code has string literals that are easy to mistype. There is a way to write this code in a less error-prone manner.

Passing JNDI parameters - 1 (variant)

```
import javax.naming.*;
import com.sun.jndi.fscontext.FSContextFactory;
import java.util.Hashtable;
public class InitWithHashtable {
  public static void main( String[] args ) throws NamingException {
      Hashtable<String,String> env = new java.util.Hashtable<>();
      env.put( Context.INITIAL_CONTEXT_FACTORY,
               FSContextFactory.class.getName() );
      env.put( Context.PROVIDER_URL, "file:///home/kto/" );
      Context ctx = new InitialContext(env);
      try { System.out.println( ctx.getEnvironment() ); }
      finally { ctx.close(); }
  }
}
```

Now we have used named constants instead of two string literals: the name of the `static final` fields of `Context` will be verified by the compiler, hence we can avoid mistyping. The name of the factory class is also passed in a less error-prone manner, by reading the name from the respective class object through the use of a class literal. A drawback of this solution is that now the class definition statically refers to the initial context factory class, and hence the two JARs that make up the service provider must be included in the classpath also for compilation. Even more embarrassing is that due to the static dependency, the code must be recompiled whenever the service provider (and hence the factory class) is changed. In the first version there was no static dependency, the code could read in the name of the factory class from a configuration file, for instance. This flexibility is often required in applications, so there are standard and easy ways to achieve this.

Passing JNDI parameters - 2

```
import javax.naming.*;
public class InitWithSystemProps {
  public static void main( String[] args ) throws NamingException {
      Context ctx = new InitialContext();
      try { System.out.println( ctx.getEnvironment() ); }
      finally { ctx.close(); }
  }
}
```

Compile and run

```
javac InitWithSystemProps.java
java -classpath fscontext.jar:providerutil.jar:.
     -Djava.naming.factory.initial=com.sun.jndi.fscontext.FSCon
                                                        textFactory
     -Djava.naming.provider.url=file:///home/kto/
     InitWithSystemProps
```

The Java code becomes much simpler, and we do not need the JARs for compiling the code. However, the command-line for running the code becomes a bit complicated: we specify the JNDI properties using the `-D` switch, which allows us to pass ``system properties'' to the JVM, and to the application running in the JVM. These system properties will be used by the JVM to automatically configure JNDI for us.

Obviously a wrapper shell script can help to avoid typing that much at the command line.

Passing JNDI parameters - 3

```
import javax.naming.*;
public class InitWithSystemProps {
  public static void main( String[] args ) throws NamingException {
      Context ctx = new InitialContext();
      try { System.out.println( ctx.getEnvironment() ); }
      finally { ctx.close(); }
  }
}
```

Compile and run

```
javac InitWithSystemProps.java
java -classpath fscontext.jar:providerutil.jar:. InitWithSystemProps
```

jndi.properties file

```
java.naming.factory.initial=com.sun.jndi.fscontext.FSContextFactory
java.naming.provider.url=file:///home/kto
```

Another solution is to use a special file to store the JNDI-specific properties. In the Java platform, *property* files appear often. They are text files containing key-value pairs (property-value bindings). Each line corresponds to such a key-value pair. Property files have the extension `.properties`.

A special property file is the `jndi.properties` file, which, if in the classpath, will be automatically loaded by the JVM when JNDI is to be initialized. Now we can save our JNDI properties in such a file, and place it for instance in the root directory of our class files (where you would put the class files in the default package).

Algorithm to set up environment for a Context

- Environment object parameter for `InitialContext` constructor
- System properties
- Applet parameters
- Application resource files

The algorithm to set up the environment of the initial context is the following. First we consider the properties in the `Hashtable` passed as the argument to the `InitialContext` constructor. Less priority is assigned to system properties, even less to applet parameters (not covered in this lesson), and finally to the properties in `jndi.properties`.

Lookup in FSContext

Assume that `java.naming.provider.url=file:///home/kto`.

```
Context root, inner;
java.io.File file;

root = new InitialContext();

file = (java.io.File) root.lookup("apple.txt");  // a file
inner = (Context) root.lookup("bananas");        // a directory
```

```
inner.lookup("/banana.txt")
inner.lookup("banana.txt")
root.lookup("bananas/banana.txt")
root.lookup("file:///home/kto/bananas/banana.txt")
```

We perform a couple of lookups here. First an initial context is created; assume that it represents the `/home/kto` directory (this can be achieved by setting the provider URL to `file:///home/kto`). Furthermore, assume that this directory contains a file `apple.txt`, a directory `bananas`, and this latter contains a file `banana.txt`. The first lookup is relative to the initial context, i.e. `root`. Hence it will find the file `/home/kto/apple.txt`. The lookup in the JNDI API statically returns a `java.lang.Object`. I have casted it to `java.io.File` because of our assumption tells me that `/home/kto/apple.txt` is indeed a file, and I also know that the File System Service Provider used here returns `java.io.File` objects when a file is looked up.

The second lookup returns another `java.lang.Object`, which identifies a (sub)directory: `/home/kto/bananas`. Since this is an inner node in the JNDI tree rooted in `root`, it will be a `javax.naming.Context` object.

Now we can perform a lookup relative to the `banana` subdirectory. The slash before the name `banana.txt` does nothing - both lookups will return a `java.io.File` object representing the file `/home/kto/bananas/banana.txt`.

The very same file can be found by a lookup relative to `root` as well. We use a compound name in the next call to `lookup`; the File System Service Provider expects name components be separated by slashes.

The last lookup is very different from the others. This is not a relative lookup; when the name passed to `lookup` is a URI (that is it has a structure `schema:something`), the lookup is considered an *absolute* lookup, and is performed differently.

Absolute lookups

- Lookup of `http://www.elte.hu/index.html`
- JNDI searches for appropriate factory class
  - `com.sun.jndi.url.<scheme>.<scheme>URLContextFactory`
  - `com.sun.jndi.url.http.httpURLContextFactory`
- Search is also based on `java.naming.factory.url.pkgs` list
  - Assume the property is set to: `test.jndi;com.mycompany`
  - `test.jndi.http.httpURLContextFactory` and `com.mycompany.http.httpURLContextFactory`

When lookup is performed on a name with the structure `scheme:something`, there is no need to specify the factory class a priory (as a system property, jndi.properties or whatsoever). Instead, the lookup will create an appropriate instance of `Context`

based on the scheme part of the URI. The factory class capable of processing the given scheme is assumed to have a specific name, as seen above.

There is a JNDI property, `java.naming.factory.url.pkgs`, that can be set to a list of package names. The packages in that list are also searched for the scheme-specific subpackage and class. If none of the classes with these names exists in the classpath, the lookup will fail.

The RMI Registry JNDI Provider

Remember: simple names, no hierarchy

```
import javax.naming.*;
public class Server {
  public static void main( String args[] ) throws Exception {
        Service service = new ServiceImpl();
        // java.rmi.Naming.bind("Toms_diner", service);
        Context ctx = new InitialContext();
        try {
            ctx.bind("rmi://localhost:1099/Toms_diner", service);
            System.out.println("Service is bound.");
            System.in.read();
            ctx.unbind("Toms_diner");
        } finally { ctx.close(); }
  }
}
```

Another widely used JNDI provider is an RMI Registry provider. This provider has no hierarchy: the *lookup* method only accepts simple names. We can see the use of the naming API of RMI in the comment. This can be replaced by the use of the JNDI API.

Federation

```
Hashtable<String,String> env = new Hashtable<>();
env.put( Context.INITIAL_CONTEXT_FACTORY,
         "com.sun.jndi.ldap.LdapCtxFactory" );
env.put( Context.PROVIDER_URL,
         "ldap://fun/dc=fun,dc=inf,dc=elte,dc=hu" );
env.put( Context.SECURITY_PRINCIPAL,
         "cn=Manager, dc=fun,dc=inf,dc=elte,dc=hu" );
env.put( Context.SECURITY_CREDENTIALS,
         "mysecretpassword" );
Context ctx = new InitialContext(env);

Context rmictx = (Context) ctx.lookup("rmi://fun.inf.elte.hu:1099");
```

```
ctx.bind("cn=registry", rmictx);

ctx.bind("cn=registry/Toms_diner", new ServiceImpl());
         // composite name, spanning multiple naming systems
```

Advanced naming service providers are able to participate in federations. A federation is a naming system that is served by multiple naming services, even by ones with different implementations. When a lookup is initiated, some name components are handled by one of the participating naming services, but when the boundary of its naming system is reached, it passes the rest of the name to the other naming service transparently.

In this specific example the root of a JNDI tree is served by an LDAP service. A subtree is bound to an RMI Registry service. The two services form a federation.

First we set up the initial context for LDAP. Then an absolute query on this context, `ctx`, is performed; remember, in this case the lookup is not served by `ctx`, but rather a new initial context for the RMI scheme is created. This rmi context is bound as a subtree in the LDAP naming system under the name `cn=registry`. Our remote object can now be bound with the composite name `cn=registry/Toms_diner` into the LDAP context, or alternatively, directly to `rmictx` with the name `Toms_diner`; the result is equivalent. Resolving the composite name involves a switch between naming services, when the boundary of the first naming system is reached.

Terminology

- Naming system: related to a particular naming service
- Naming convention: rules of a naming system
- Namespace: set of the valid names within a naming system
- Name: element of the namespace, e.g. `/home/kto/bananas/banana.txt`
- Atomic name: indivisible in the particular naming convention, e.g. `kto`
- Compound name: when the namespace is hierarchical
  - Components: `home`, `kto`, `bananas`, `banana.txt`
  - Components of `www.elte.hu` are `hu`, `elte`, `www` (note the order!)
- Composite name: spans multiple naming systems (federation)
  - `www.elte.hu/home/kto/bananas/banana.txt`